

CareerForge

Candidate Review Ops Console

Day 2 Incident Analysis Report

The System Behind The Bug

Name	Vibhut Vishal Uttarde
Variant ID	D2-A — Database Write Latency
College	Vidyalankar Polytechnic
Branch / Year	Information Technology / 3rd Year
Decision Moment	11:00 AM
Date	09 June 2026

Repository

github.com/vibhututtarde/day-2-The-System-Behind-The-Bug

Upload this document + 4-minute reasoning video to the repo above

Section 1 — Incident Reasoning Brief

Variant ID: D2-A | Product: Candidate Review Ops Console | Decision Moment: 11:00 AM

1. Founder Assumption

What the founder assumed:

"This looks like a notification or recruiter action issue. Fix that first."

Verdict: Misleading.

The founder looked at the most visible and noisy symptoms — recruiter complaints and notification delays — and named them as root causes. This is the classic error of treating the loudest downstream effect as the origin failure. The evidence shows notifications are delayed because database writes have not committed yet. Fixing notifications first would not stop write latency, and all symptoms would continue.

2. Competing Hypotheses

Hypothesis	Evidence Supporting It	Evidence Against It	Verdict
H1: Notification queue is the root cause	Notification delay is 70–150s (vs. normal 5–20s). Candidates received emails but portal still showed "Under Review."	Log at 10:01:13 shows "Candidate status record not committed yet" before the notification delay is logged at 10:01:22. Notifications are delayed because they are waiting on a status that has not committed.	REJECT — Downstream symptom.
H2: PostgreSQL write latency is the root cause	Write latency spiked from 80–250ms to 3,500–7,800ms (14–31× normal). Logged at 10:01:10. Traffic spiked 5.7× (90 → 510 actions/min).	No evidence of a database crash, corrupt data, or infrastructure failure. Latency is a throughput-under-load pattern, not a software bug.	ACCEPT — Origin failure.
H3: Recruiters are double-clicking (user error)	Duplicate shortlist reports rose from 1–2/hr to 39/hr. Looks like accidental repeated submissions.	Log at 10:01:14: retry triggered by the system because the write had not confirmed — not a human clicking twice. The UI showed no success state because the DB had not committed.	REJECT — System-induced, not user behaviour.

Hypothesis	Evidence Supporting It	Evidence Against It	Verdict
H4: Redis / Dashboard cache is the root cause	Dashboard lag is 1–4 min (vs. <5s normal). Shortlist count jumped before DB status caught up. Redis mismatch logged at 10:01:20.	Redis is reading stale data because the DB committed late. Redis cannot show what PostgreSQL has not written. The lag is a dependent effect.	REJECT — Downstream symptom.

3. Origin Failure

What failed first: PostgreSQL write latency under high-volume load.

Confidence: High

Why:

- The DB write latency WARN (10:01:10, duration_ms=5100) appears in the log before every other downstream failure: the "not committed yet" warning (10:01:13), the retry (10:01:14), the duplicate (10:01:16), the Redis mismatch (10:01:20), the notification delay (10:01:22), the dashboard sync delay (10:01:23), and the analytics mismatch (10:01:30).
- The causal chain is unambiguous: everything downstream requires a committed write. If the write is slow, everything that depends on it — notifications, cache, dashboard, analytics — either waits, reads stale data, or receives out-of-order events.
- The traffic spike (5.7× above normal) provides a concrete load-based explanation for why writes degraded with no other infrastructure change required.

Supporting Evidence:

- E1: Log 10:01:10 — WARN DB write latency high table=candidate_status duration_ms=5100 — first failure signal in the log sequence.
- E2: Metrics — PostgreSQL write latency jumped from 80–250ms (normal) to 3,500–7,800ms (current), a 14–31× increase.
- E3: Log 10:01:13 — WARN Candidate status record not committed yet candidate_id=c_2209 — the uncommitted write is the proximate cause of every downstream symptom.
- E4: Metrics — Traffic jumped from 90 to 510 actions/minute (5.7× increase), beginning at 10:00 AM, consistent with a throughput-induced write bottleneck.
- E5: Analytics: 16% of shortlist events have no matching committed status — exactly what you would see if analytics consumed click events while the DB write was still in flight.

4. Symptom vs Root Cause Table

Reported Issue	Root Cause / Downstream Effect	Evidence
Recruiter actions not updating	Downstream. Write latency means status changes do not commit for 3.5–7.8s instead of <250ms. The action was accepted but not yet persisted.	Log 10:01:09 + 10:01:10 + 10:01:19
Duplicate shortlist actions	Downstream. No confirmation returned to UI because write had not committed. System triggered retry at 10:01:14.	Log 10:01:14 + 10:01:16; retry rate rose from 1.5% to 13.6%
Notification delays	Downstream. Notification queue requested event while DB still showed candidate as uncommitted. Queue waited for committed state, adding 70–150s delay.	Log 10:01:12 + 10:01:13 + 10:01:22
Candidate status mismatch	Downstream. Redis was updated optimistically on action receipt, before DB committed.	Log 10:01:20 — Redis shows shortlisted, DB shows pending_commit
Dashboard lag	Downstream. Dashboard reads from Redis (stale) and DB (slow). Both sources degraded.	Metrics: dashboard lag 1–4 min (normal <5s)
Analytics inconsistency	Downstream. Analytics received shortlist_clicked events but no matching candidate_status_committed event existed. 16% mismatch.	Log 10:01:29 + 10:01:30

5. Failure Propagation Chain

ORIGIN FAILURE:

PostgreSQL write latency spikes to 3.5–7.8s under 5.7x traffic load
(Log 10:01:10, Metrics: 80–250ms → 3,500–7,800ms)

↓

Candidate Review Service writes status change; DB accepts the row but commits slowly — candidate_status_committed fires late
(Log 10:01:13: status record not committed yet)

↓

BRANCH A: UI has no confirmation → system triggers retry
→ Duplicate shortlist actions submitted
→ Duplicate shortlist reports spike: 1-2/hr → 39/hr

↓

BRANCH B: Redis updated optimistically before DB commit resolves
→ Redis shows shortlisted; DB still shows pending_commit
→ Status mismatch visible to candidates and support

```

↓
BRANCH C: Notification queue fires against uncommitted status
→ Queue stalls until commit resolves
→ Notification delay: 5-20s → 70-150s

↓
BRANCH D: Dashboard reads stale Redis + uncommitted DB state
→ Shortlist counts incorrect; lag grows to 1-4 min

↓
BRANCH E: Analytics consumes click events before status commits
→ 16% of events have no matching committed status

↓
BUSINESS IMPACT:
Recruiters lose confidence and retry → more write load (feedback loop)
Hiring operations cannot trust shortlist counts at deadline
Candidates receive contradictory information

```

6. First 15-Minute Incident Actions

Action	Why Now	Risk Reduced	Tradeoff
1. Throttle / pause the review sprint	DB is under 5.7× normal load. Every new write action worsens the backlog and generates more retries.	Stops the retry feedback loop. Prevents new duplicates. Gives DB time to drain the write queue.	Recruiters cannot review candidates for ~10 min. Acknowledged delay is better than data corruption.
2. Identify and deduplicate shortlist records	Duplicate entries are already in the DB. If hiring ops exports the shortlist now, they will get inflated counts.	Protects the shortlist from being acted on with wrong data at the deadline.	Requires a fast DB query and manual review — small engineering cost, high business value.
3. Communicate status to hiring ops and recruiters	Recruiters retrying actions are the active amplifier of the duplicate problem. A clear message stops retries immediately.	Immediately reduces retry rate (currently 13.6% vs. 1.5% normal). Prevents further duplicate creation.	None — pure risk reduction with no tradeoff.

7. What I Would Not Fix Immediately

Delayed Work	Why Delay	Risk Accepted
Notification queue delays	Notifications are delayed, not lost. Fixing the queue now while writes are still backed up would send notifications against stale data,	Candidates experience longer notification delays for ~10–20 min.

Delayed Work	Why Delay	Risk Accepted
	making the mismatch worse.	
Dashboard / Redis reconciliation	Once DB writes drain, Redis will re-sync naturally. Manually flushing Redis during active write congestion risks serving empty state to recruiters.	Dashboard shows lagged data for the recovery window.
Analytics backfill	The 16% mismatch is a reporting problem, not an operational one. Analytics can be reconciled post-incident from committed DB records.	Analytics report will be inaccurate during the incident window.

8. Tradeoff Ownership

The tradeoff accepted:

Pausing recruiter work for approximately 10 minutes to let write latency recover and verify shortlist data integrity.

Who is helped first:

Hiring operations. A clean, accurate shortlist at the deadline is more important than 10 minutes of recruiter throughput.

Who still experiences pain temporarily:

Recruiters are paused. Candidates still see delayed notifications and mismatched portal status. Both self-correct once DB write latency returns to normal.

What is explicitly not accepted:

Letting recruiters continue working through the backlog. The retry rate (13.6% vs. 1.5%) shows that recruiter behaviour is now a load amplifier on the bottlenecked DB.

9. AI / Assumption Challenge

The founder assumed: Notification delays and recruiter action issues are the root cause — fix those first.

What the evidence showed:

The logs prove that notifications failed because the DB had not committed yet. Notification delay (10:01:22) appears 12 seconds after the DB write latency warning (10:01:10) and 9 seconds after the "not committed yet" warning (10:01:13). Fixing notifications first would have no effect on DB write speed.

The misleading evidence that could have trapped me:

Notification delays are the loudest user-facing complaint. They generate the most visible support tickets. A first instinct would see "notification delayed, recruiter action broken" and jump to fixing the notification queue. The notification delay is noisy precisely because candidates see it immediately, while the DB write latency is silent infrastructure.

What changed my mind:

The log sequence. Every single downstream failure has a causal parent tracing back to the DB write latency warning at 10:01:10. There is no log evidence of a notification system failure, a queue crash, or a recruiter UI bug that predates the DB warning. The sequence is deterministic: slow write → no confirmation → retry → duplicate. Slow write → notification waits for commit → delay.

10. Final Incident Judgment

Final Judgment:

PostgreSQL write latency — driven by a 5.7× traffic spike from 90 to 510 actions/minute — is the origin failure. Every reported symptom (duplicate shortlists, notification delays, status mismatch, dashboard lag, analytics inconsistency) is a downstream effect of writes not committing within the expected window. In the first 15 minutes, pause the review sprint to reduce write load, identify existing duplicate shortlist records, and tell recruiters not to retry. Do not touch notifications or analytics until writes recover.

Final Checklist

- ✓ Challenged the founder assumption.
- ✓ Evaluated at least two hypotheses (evaluated four).
- ✓ Separated symptoms from root cause.
- ✓ Cited exact evidence (log lines + metric values).
- ✓ Explained first 15-minute actions.
- ✓ Named what to delay and why.
- ✓ Explained tradeoffs.
- ✓ Included AI / assumption verification.

One-line realization:

I initially thought the notification delay was the obvious place to start, but the evidence showed that every symptom — including notifications — was caused by a single upstream database write bottleneck that made all the downstream noise inevitable.

Section 2 — Failure Propagation Analysis

1. Origin Failure

What most likely failed first:

PostgreSQL write latency degraded severely under high-volume load. The candidate_status table writes spiked from 80–250ms (normal) to 3,500–7,800ms when review traffic jumped 5.7× from 90 to 510 actions/minute at the start of the hiring deadline sprint.

Evidence:

- Log 10:01:10 — WARN DB write latency high table=candidate_status duration_ms=5100 — first WARN in the entire log sequence, appearing before every downstream failure.
- Metrics: PostgreSQL write latency 3,500–7,800ms vs. normal 80–250ms (14–31× above normal).
- Metrics: Traffic at 510 actions/min vs. normal 90 — a 5.7× spike beginning at 10:00 AM, directly preceding the first latency warning.

Confidence level: High

The DB write latency warning precedes every other failure in the log. No other component failure appears before it. Every downstream symptom requires a committed DB write to resolve correctly, and all failed because that write was slow.

2. Propagation Chain

```
Origin failure: PostgreSQL write latency spikes to 3,500-7,800ms under 5.7x traffic load
->
System effect: Candidate status write accepted but not committed - no success confirmation returned
->
System effect: Frontend receives no confirmation -> system triggers retry (10:01:14)
->
System effect: Redis updated optimistically before DB commits -> cache shows 'shortlisted',
                DB shows 'pending_commit' (10:01:20)
->
System effect: Notification queue fires against uncommitted status -> stalls 70-150s (10:01:22)
->
Downstream symptom: Duplicate shortlist actions created by retries - 1-2/hr -> 39/hr (10:01:16)
->
```


Downstream symptom: Dashboard reads stale Redis + slow DB → lag grows to 1–4 min; counts wrong

->

Downstream symptom: Analytics receives click events without matching committed status events

— 16% mismatch

->

User / business impact: Recruiters lose trust and retry further, amplifying DB load.

Shortlist data contains duplicates. Hiring ops cannot confidently export at deadline.

Candidates receive contradictory status information.

3. Symptom Separation

Symptom	Direct Cause	Downstream From	Evidence
Recruiter actions not updating	DB write not committed within expected window — no confirmation returned to UI.	PostgreSQL write latency	Log 10:01:09 + 10:01:10 + 10:01:19
Duplicate shortlist actions	System retry fired because no confirmation was received before timeout.	PostgreSQL write latency → no confirmation → retry	Log 10:01:14 + 10:01:16; retry rate 1.5% → 13.6%
Notification delays	Queue requested status at 10:01:12 but status was not yet committed at 10:01:13 — queue stalled.	PostgreSQL write latency → uncommitted status	Log 10:01:12 + 10:01:13 + 10:01:22; delay 5–20s → 70–150s
Candidate status mismatch	Redis updated on action receipt before DB committed — cache and DB showed different states.	PostgreSQL write latency → Redis/DB divergence	Log 10:01:20: Redis=shortlisted, DB=pending_commit
Dashboard lag	Dashboard reads from Redis (stale) and DB (slow to commit) — both sources degraded.	PostgreSQL write latency → Redis mismatch + slow DB reads	Metrics: lag <5s → 1–4 min
Analytics inconsistency	Analytics consumed shortlist_clicked events before candidate_status_committed events existed.	PostgreSQL write latency → commit events arriving late	Log 10:01:29 + 10:01:30; 16% mismatch

4. Business Damage

Who is affected first:

Recruiters — they are the first to see unresponsive actions and no confirmation. This causes them to retry, which worsens DB load. By 10:12, candidate status mismatch complaints begin. By 10:25, hiring operations is affected as dashboard counts become unreliable for deadline tracking.

What gets worse if no one acts:

The retry feedback loop is the critical risk. Every retry is a new write submitted to an already overloaded DB. The more recruiters retry, the higher the write volume, the longer the latency stays elevated, the more retries are triggered. Left unchecked, duplicate shortlist records accumulate throughout the sprint window, meaning the final shortlist export at the deadline will contain inflated, incorrect candidate counts. Hiring decisions get made on corrupt data.

What evidence proves the impact:

- Duplicate shortlist reports at 39/hr (normal 1–2/hr) — data integrity is already compromised.
- Retry rate at 13.6% (normal 1.5%) — the feedback loop is active and growing.
- 16% analytics mismatch — the deadline report cannot be trusted in its current state.
- Hiring ops message at 10:40: "Should recruiters keep working?" — operational confidence has collapsed.

5. Final Propagation Judgment

Summary:

PostgreSQL write latency, triggered by a 5.7× traffic spike, was the single origin failure. Because every downstream service — the notification queue, Redis cache, dashboard, and analytics — depends on a committed DB write to function correctly, slow writes caused all of them to either stall, read stale data, or receive out-of-order events simultaneously. Recruiters, seeing no confirmation, triggered system retries that amplified the original load and created duplicate shortlist records. What looked like five separate problems was one failure seen from five different angles.

Section 3 — Incident Prioritization Artifact

1. First Stabilization Priority

What should be stabilized first:

First Action:

Reduce write load on PostgreSQL by pausing the recruiter review sprint and stopping the retry feedback loop.

Why this first:

Every other symptom — duplicates, notification delays, status mismatch, dashboard lag, analytics inconsistency — is caused by slow DB writes. Nothing downstream can recover while write latency stays at 3,500–7,800ms. The retry loop (recruiter actions generating repeated writes because no confirmation is returned) is actively worsening the load. Cutting write volume is the only action that addresses the root cause without a code change and gives the DB a chance to drain its backlog.

Evidence:

- Log 10:01:10 — WARN DB write latency high table=candidate_status duration_ms=5100 — first failure in the log; all other failures are downstream of this.
- Metrics: traffic spiked 5.7× (90 → 510 actions/min); recruiter retry rate rose from 1.5% to 13.6%, meaning retries are adding ~61 extra write operations per minute on top of already excessive load.
- Log 10:01:14 — INFO Recruiter action retry started — system-triggered, not user error; retries are structural until write latency resolves.

2. Delay List

Work To Delay	Why Delay It	Risk Accepted
Notification queue fix	Notifications are delayed, not lost. They will drain automatically once DB writes recover and statuses are committed. Intervening now risks sending notifications against uncommitted status, worsening the candidate portal mismatch.	Candidates experience delayed notifications for the recovery window (~15–30 min total).
Redis / dashboard reconciliation	Redis will re-sync on next read after DB commits resolve. Forcing a manual Redis flush during an active write backlog risks serving empty or blank state to recruiters, which is more disruptive than a lagged count.	Dashboard shows stale shortlist counts until recovery. Acceptable since recruiters are paused.

Work To Delay	Why Delay It	Risk Accepted
Analytics backfill	The 16% analytics mismatch is a reporting problem, not an operational one. The shortlist — not the analytics report — is what hiring ops uses for deadline decisions. Analytics can be reconciled from committed DB records after stabilization.	Analytics report is unreliable during the incident window. Hiring ops must be told not to use it for decisions until reconciled.

3. First 15-Minute Plan

Minute Window	Action	Risk Reduced	Tradeoff
0–5 min	Tell hiring ops to pause the review sprint. Tell all recruiters: "Your actions were received — do not retry."	Stops new writes from entering the backlog. Breaks the retry feedback loop immediately. Prevents additional duplicate shortlist records from being created.	Recruiters cannot process candidates for ~10 min. Acknowledged pause is better than corrupted shortlist data at deadline.
5–10 min	Query candidate_status and shortlist tables for candidates with more than one shortlist event between 10:00–11:00. Flag them — do not delete yet.	Identifies existing data integrity damage before it reaches the deadline export. Prevents hiring ops from acting on an inflated shortlist count.	Requires engineering time (~5–10 min). No user-facing risk from the query itself.
10–15 min	Monitor PostgreSQL write latency. Wait for confirmed return below 500ms before allowing recruiters to resume. Brief hiring ops on what happened and that the shortlist needs review before export.	Prevents the feedback loop from restarting too early. Gives hiring ops the information they need to hold the deadline export until data is clean.	Recruiters remain paused until confirmation. Extends downtime by a few minutes but prevents a second incident.

4. Stakeholder Impact

Who benefits from this priority:

Hiring operations benefits first and most. Pausing the sprint and identifying duplicates protects the shortlist data integrity before the deadline export. A clean shortlist matters more than 10 minutes of recruiter throughput. Recruiters also benefit indirectly — their actions taken between 10:00–11:00 were received and will be correctly committed once latency recovers; they will not lose work.

Who still experiences pain temporarily:

Recruiters are paused and cannot process candidates during the recovery window. Candidates continue to see delayed notifications and mismatched portal status until the notification queue drains after DB recovery. Neither of these is a data integrity risk — they are experience problems that resolve automatically once writes stabilize.

5. Final Priority Defence

Defence:

The root failure is PostgreSQL write latency, not notifications or recruiter actions. Fixing notifications or the dashboard while writes are still backed up would deliver notifications against uncommitted state and make the mismatch worse. The only correct first move is to reduce the write volume driving the latency — which means pausing the sprint and stopping retries. This costs 10 minutes of recruiter time but protects the one thing that cannot be wrong at a hiring deadline: the shortlist itself. Duplicate records already created in the 10:00–11:00 window must be identified before any export happens. Everything else — notifications, Redis, analytics — recovers automatically once DB writes return to normal, and delaying fixes to those components carries no additional data integrity risk.